

Lab: Extreme Events

Samuel Harris

Modeling the Unusual

- For a given sample, the maximum and minimum are fixed
- But if we have a large number of samples, we can study the distribution/density of those max/min draws
- Imagine:
 - The biggest storm in a year
 - The worst loss for the market in a month
 - The biggest earthquake in a decade
 - The longest time between goals in a game
- This is how we understand the distribution/density of extreme events
- The **k -th order statistic** is the k -th largest value in a sample of n from some distribution
- The minimum is the first order statistic and the maximum is the n -th order statistic

Example: Drawing from a Gaussian

Below, we find the minimum of a sample of size n drawn from a normal distribution, and then repeat this b times, to get a distribution of the minimum with 10000 counts in it. This is the distribution of the 1st order statistic, shown below in blue.

```
In [55]: # !pip install pyarrow fastparquet
```

```
In [56]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

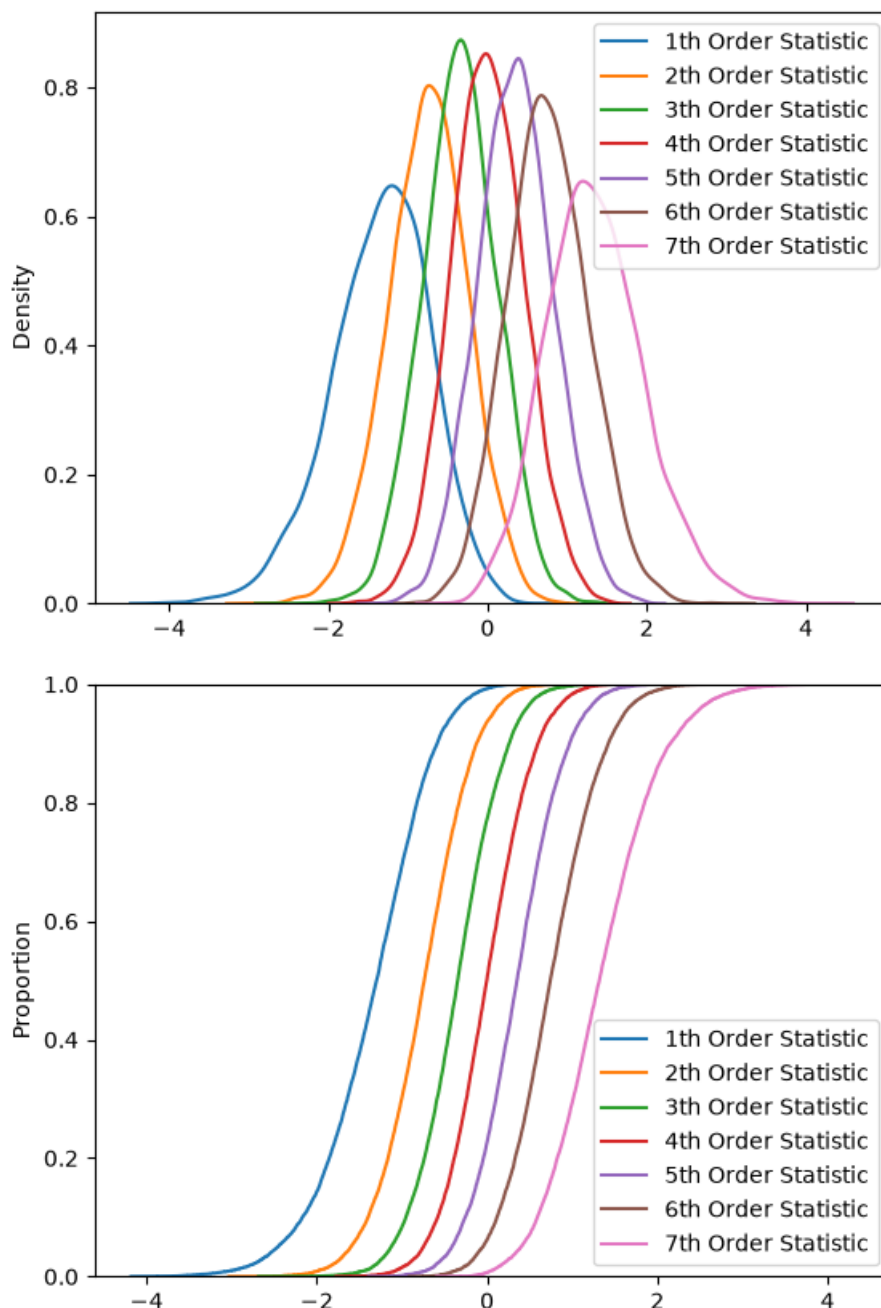
rng = np.random.default_rng( seed = 100 ) # Create a random number stream

n = 7 # Sample Size
b = 10_000 # Number of Simulations

sample = rng.standard_normal( size = (n,b) ) # Draw a sample from the standard normal distribution
sorted_sample = np.sort(sample, axis=0) # Sort each column (each simulated sample of size n)

for j in range(n): # For each rank
    sns.kdeplot( sorted_sample[j,:], label=f'{1+j}th Order Statistic') # Plot the KDE
plt.legend() # Add Legend
plt.show()

for j in range(n): # For each rank
    sns.ecdfplot( sorted_sample[j,:], label=f'{1+j}th Order Statistic') # Plot the ECDF
plt.legend() # Add Legend
plt.show()
```



Analytic Relationship

Maximum

If $F(x)$ is the underlying true CDF, let's consider the analytical equation for the distribution of the maximum of a sample of size n . (The maximum is its own random variable, with its distribution plotted above. What is this in math?) To build the CDF of the maximum, we ask (as always): "What's the probability the maximum lands at or below x ?" for every x . In other words:

$$F_{max}(x) = P(X_{max} \leq x)$$

This is nothing new, just applying it to the distribution of a specific order statistic, the max.

So what is the **probability that all of the draws are less than or equal to x** , i.e. what is the probability that the maximum is less than or equal to x (the right hand side)? Because each draw is independent:

$$P(X_1 \leq x \dots, X_n \leq x) = P(X_1 \leq x) \cdot P(X_2 \leq x) \dots P(X_n \leq x)$$

Because we are pulling from the same distribution every time, **each of these probabilities equals our CDF, $F(x)$** :

$$P(X_1 \leq x, \dots, X_n \leq x) = F(x) \cdot F(x) \dots F(x)$$

The probability that all of the draws are less than or equal to x is therefore:

$$F_{\max}(x) = F(x) \cdot F(x) \dots F(x) = F(x)^n$$

What is its density?

$$f_{\max}(x) = nF(x)^{n-1}f(x)$$

Minimum

The distribution of the minimum is somewhat harder, because our CDF is asking $\leq x$, and " $\max \leq x$ " means "all $\leq x$ " but " $\min \leq x$ " does not mean "all $\leq x$ ". We can't just substitute in multiple copies of $F(x)$. Instead of looking at the probability that all draws are less than or equal to x , we need to know the probability that at least one is.

We want to ask "is the minimum less than or equal to x ?" which is the opposite of asking "are all of the draws, including the minimum, above x ?" The probability that all observations are above x is $(1 - F(x))^n$. Because this is the opposite of the question we actually want to ask, the complementary probability is what we truly want:

$$F_{\min}(x) = 1 - (1 - F(x))^n$$

What is its density?

$$f_{\min}(x) = nf(x)(1 - F(x))^{n-1}$$

All of the other order statistics, generally

- If you're curious, the density of the k -th of n order statistics is

$$f_k(x) = \frac{n!}{(k-1)!(n-k)!} F(x)^{k-1} (1 - F(x))^{n-k} f(x)$$

- We'll use the results for the minimum and maximum

The Data

We have three datasets:

1. Rainfall *totals* for every state-month from 1981 to 2025, with 5 reporting stations per state each month
2. Rainfall for every *station-day* over that same 1981-2025 period (same 5 stations per state) — much bigger, and used for a single question near the end of Step One
3. Daily and monthly U.S. stock market returns from 1926 to 2026

We want to understand tail behavior, to better reason about risk (engineering, financial, etc.).

```
In [57]: df = pd.read_parquet("data/ghcn_prcp_monthly.parquet")
df.head()
```

```
Out[57]:
```

	state	state_name	station_rank	station_id	station_name	latitude	longitude	elevation_m	year	month	month_name
0	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156.7394	8.2	1981	1	January
1	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156.7394	8.2	1981	2	February
2	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156.7394	8.2	1981	3	March
3	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156.7394	8.2	1981	4	April
4	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156.7394	8.2	1981	5	May

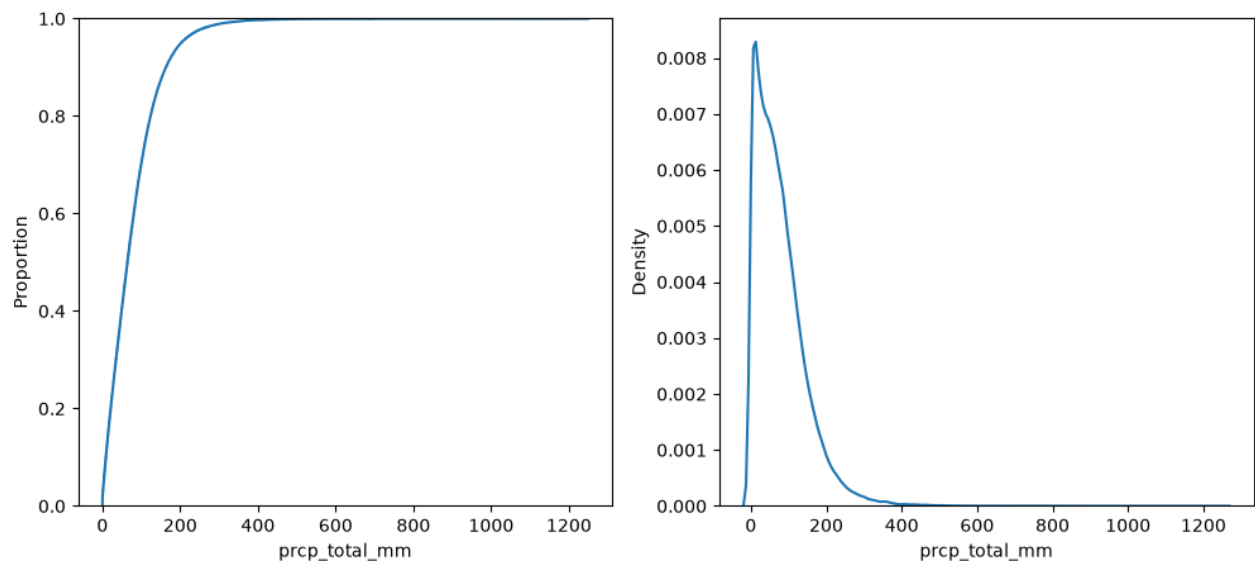
Step One.

1. Plot the ECDF and KDE of `prcp_total_mm` with seaborn— the distribution and density of rainfall across the entire U.S.

```
In [58]: fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(12, 5))

sns.ecdfplot(df, x='prcp_total_mm', ax=ax[0])
sns.kdeplot(df, x='prcp_total_mm', ax=ax[1])
```

```
Out[58]: <Axes: xlabel='prcp_total_mm', ylabel='Density'>
```



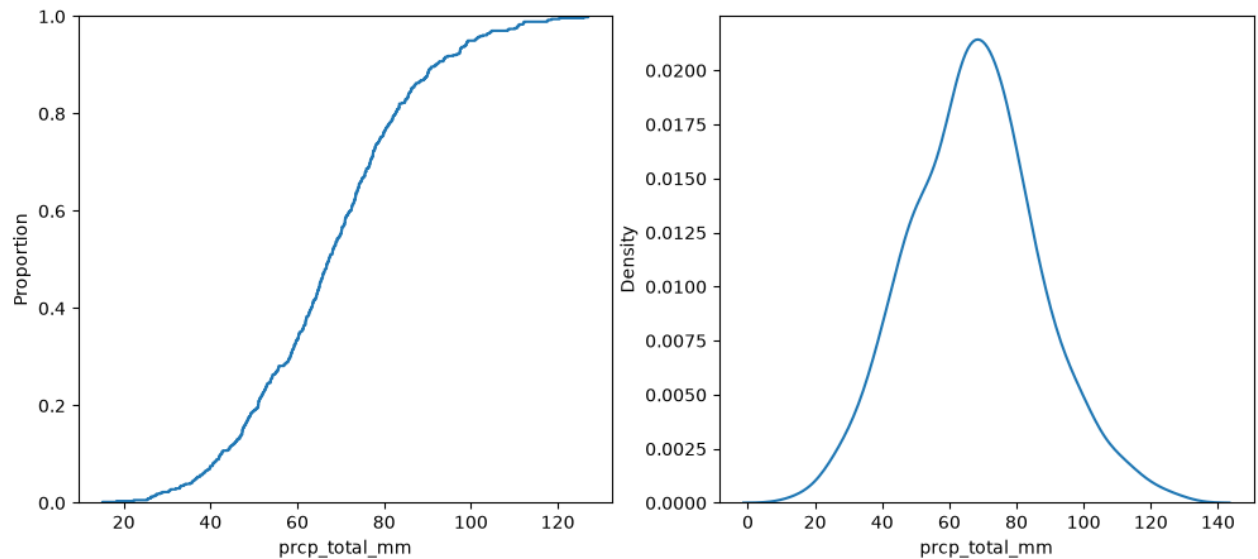
2. Now group by year-month and compute the **median** and **maximum** precipitation nationwide. Plot the ECDF and KDE of these two statistics.

```
In [59]: median = df.groupby(['year', 'month'])['prcp_total_mm'].median().reset_index()
maximum = df.groupby(['year', 'month'])['prcp_total_mm'].max().reset_index()

fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(12, 5))

sns.ecdfplot(median, x='prcp_total_mm', ax=ax[0])
sns.kdeplot(median, x='prcp_total_mm', ax=ax[1])
```

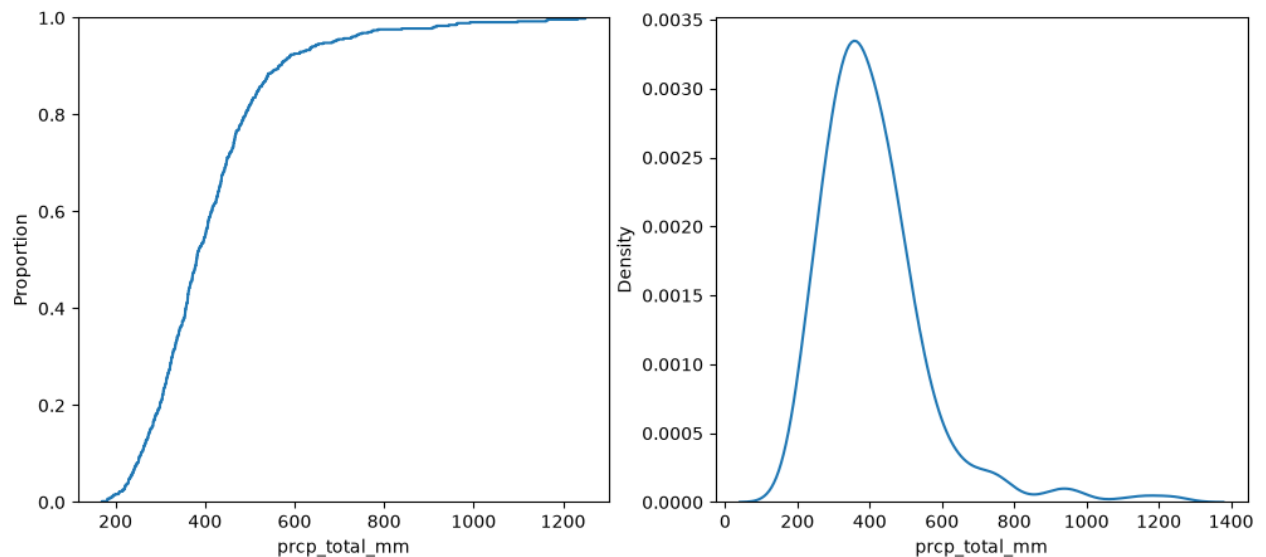
```
Out[59]: <Axes: xlabel='prcp_total_mm', ylabel='Density'>
```



```
In [60]: fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(12, 5))
```

```
sns.ecdfplot(maximum, x='prcp_total_mm', ax=ax[0])
sns.kdeplot(maximum, x='prcp_total_mm', ax=ax[1])
```

```
Out[60]: <Axes: xlabel='prcp_total_mm', ylabel='Density'>
```



3. How do your results in part 2 compare to your results in part 1?

Compare and contrast the ECDFs and KDEs from parts 1 and 2. Explain why they are different:

The overall rough shape ECDF and KDE plots for part 2 are smoother than the initial plots from part 1. The ECDF plots for part 2 have less data, and so the actual small details of the ECDF lines are a bit more jagged than the ECDF from part one which has all of the data.

The range for the X axes for the plots in part 2 are also different, as the plots for the median are concentrated towards the initial large spike in the first KDE plot.

The KDE plot for the median values seems to be more normally distributed than the other two plots, and the maximum plots from part 2 follow the shape of the initial plots more closely than the median plots.

4. Now define a state's rainfall differently: take the **median** of the 5 stations in each month, then find the **maximum** across the state's 12 months in a year — the "wettest month." For each state, what's the expected value of the wettest month? Plot the ECDF of Virginia's annual maxima (the wettest month in each of the 45 years), and overlay the analytical prediction $F_{\max}(x) = \hat{F}(x)^{12}$.

```
In [61]: # One rainfall value per state-month, taking the median of the stations
state_month = df.groupby(['year', 'month', 'state'])['prcp_total_mm'].median().reset_index()

# Maximum of the 12 state-month observations above, for each year: the wettest month of that year
state_year_max = state_month.groupby(['year', 'state'])['prcp_total_mm'].max().reset_index()

exp_wettest = state_year_max.groupby("state", as_index=False).agg(E_wettest_month=("prcp_total_mm", "mean"))
exp_wettest.loc[exp_wettest["state"] == "VA"]

state = "VA"

levels = (
    state_month.loc[state_month["state"] == state, "prcp_total_mm"]
    .to_numpy()
)

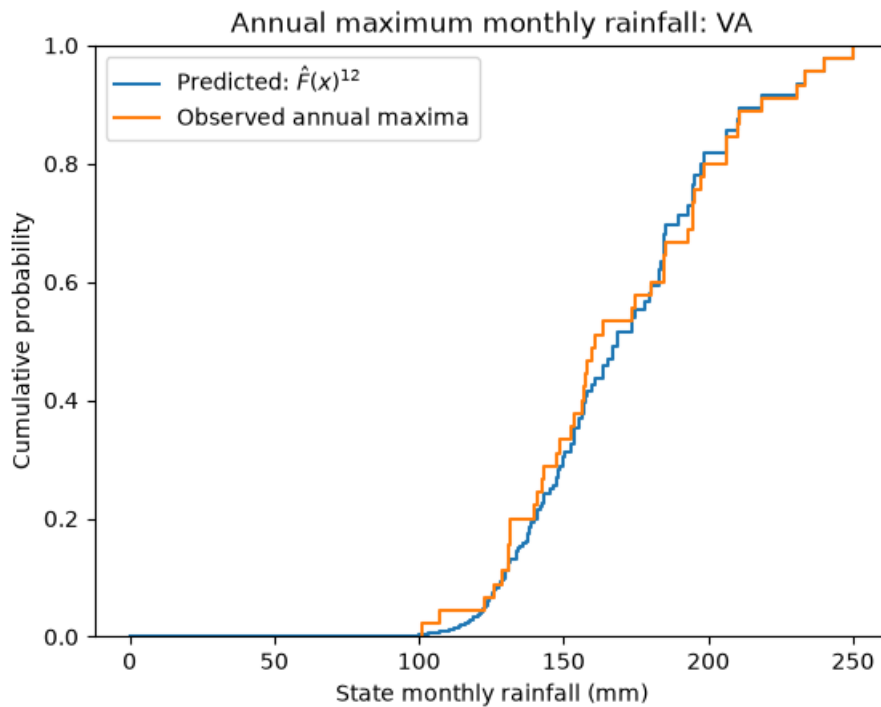
observed_max = (
    state_year_max.loc[state_year_max["state"] == state, "prcp_total_mm"]
    .to_numpy()
)

x = np.sort(np.unique(levels))
F_hat = np.searchsorted(np.sort(levels), x, side="right") / len(levels)

plt.step(
    x,
    F_hat**12,
    where="post",
    label=r"Predicted: $\hat{F}(x)^{12}$"
)

sns.ecdfplot(x=observed_max,
    label="Observed annual maxima")

plt.xlabel("State monthly rainfall (mm)")
plt.ylabel("Cumulative probability")
plt.title(f"Annual maximum monthly rainfall: {state}")
plt.legend()
plt.show()
```



How close is the fit here? If it differs from question 4's fit, why:

The fit seems to be very close. The observed curve starts off being slightly above the predicted, but goes below further along, and overall seems to hug it tightly.

5. **A question for FEMA.** So far you've worked with *monthly* rainfall totals. Switch now to the daily station-level file (`ghcn_prdp_daily.parquet`) — reload `df` from it, and don't mix it with the monthly data above.

A day with more than 120mm of rain is a serious flood/landslide risk. On any single station-day, that's rare. But a state doesn't get one chance a month — it gets $5 \text{ stations} \times (\text{days in the month})$ chances. FEMA doesn't care about the average day, or the single worst day on record; it cares about the probability that *some* station has a disaster day at some point during the month.

For each state-month, estimate

$$P(\text{at least one station-day exceeds 120mm}) = 1 - \hat{F}(120)^n$$

where $\hat{F}$ is the empirical CDF of a single station-day's rainfall for that state and calendar month (pooled across all years), and n is the number of station-days in one occurrence of that month ($5 \text{ stations} \times \text{days in the month}$). Report the 20 highest-risk state-months.

```
In [62]: # Fresh load -- this question uses only the daily file, not the monthly one above.
#df_daily = pd.read_parquet("data/ghcn_prdp_daily.parquet")

url = (
    "https://www.dropbox.com/scl/fi/"
    "pjmyop3fqjdy8oabkc4ab/ghcn_prdp_daily.parquet"
    "?rlkey=otxenczb3pxgzz53kcli5zxww&dl=1"
)
df_daily = pd.read_parquet(url)
df_daily = df_daily.loc[df_daily["observed"]]

threshold = 120 # mm in a single day

def disaster_risk(group):
    F_hat = (group["prcp_mm"] <= threshold).mean()
```

```

n = len(group) / group["year"].nunique() # station-days in one year's occurrence of this month
return 1 - F_hat**n

risk = (
    df_daily.groupby(["state", "month"])
        .apply(disaster_risk, include_groups=False)
        .rename("p_any_station_day_exceeds_120mm")
        .reset_index()
        .sort_values("p_any_station_day_exceeds_120mm", ascending=False)
)

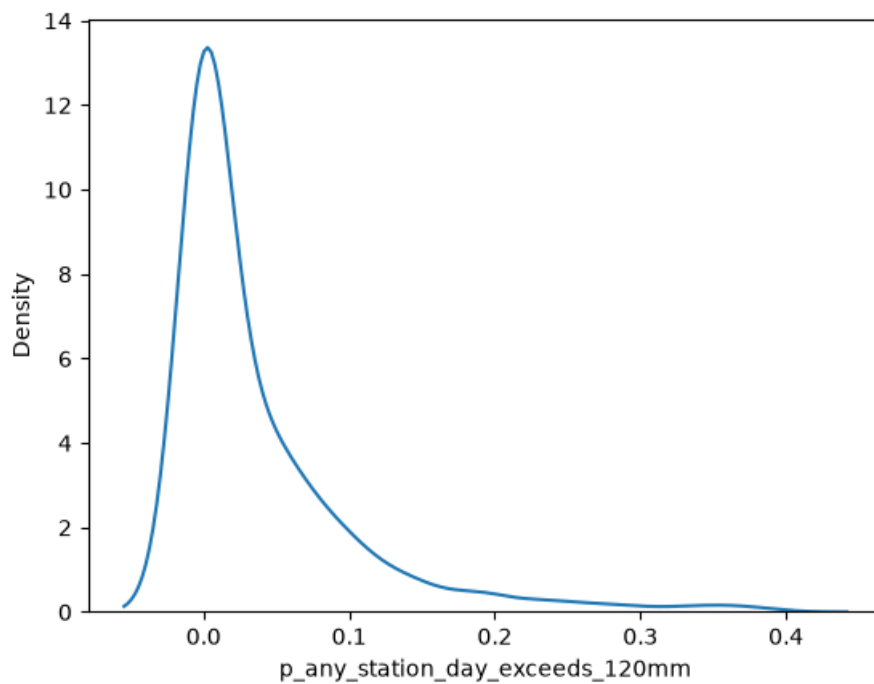
sns.kdeplot(risk['p_any_station_day_exceeds_120mm'])

risk.head(20)

```

Out[62]:

	state	month	p_any_station_day_exceeds_120mm
122	HI	3	0.387166
104	FL	9	0.373367
130	HI	11	0.359242
208	LA	5	0.359229
131	HI	12	0.344789
121	HI	2	0.330060
105	FL	10	0.330026
476	SC	9	0.299512
207	LA	4	0.299512
319	NC	8	0.283726
101	FL	6	0.267604
536	VA	9	0.267604
211	LA	8	0.267597
209	LA	6	0.251113
212	LA	9	0.234253
127	HI	8	0.234248
215	LA	12	0.234248
103	FL	8	0.234248
117	GA	10	0.234248
120	HI	1	0.217012



Which states and months face the most risk of a greater-than-120mm day? How high is the probability? Among all state-months, describe the density of high precipitation days:

Hawaii, Florida and Louisiana seem to face the most risk of a greater than 120mm day. The highest probability for a station in a state facing a high precipitation day at some point in a month is 0.387, which is the probability the a station in Hawaii will face a high precipitation day at some point during the month of March. Most state-month combos seem to have a relatively low chance of experiencing a high precipitation day at one of their stations, which the peak of the KDE plot being close to 0. As $p_{\text{any_station_day}} > 120\text{mm}$ increases, the density of state-month combos falls off quickly, with a very small number of state-month combos persisting from 0.2 to 0.4.

Step Two.

Our second data set is daily and monthly U.S. stock market returns. Instead of storms across space (five stations in a state) or across the calendar (twelve months in a year), the "sample" is the set of trading days within a month, and we ask about the best and worst day in that sample.

```
In [63]: df = pd.read_parquet("data/ff_market_daily.parquet")

print(f"{len(df):,} trading days")
print(f"{df['date'].min().date()} through {df['date'].max().date()}")
df.head()
```

26,296 trading days
1926-07-01 through 2026-07-31

```
Out[63]:
```

	date	year	month	month_start	trading_day_of_month	mkt_excess_return_pct	risk_free_return_pct	market_
0	1926-07-01	1926	7	1926-07-01	1	0.09	0.01	
1	1926-07-02	1926	7	1926-07-01	2	0.45	0.01	
2	1926-07-06	1926	7	1926-07-01	3	0.17	0.01	
3	1926-07-07	1926	7	1926-07-01	4	0.09	0.01	
4	1926-07-08	1926	7	1926-07-01	5	0.22	0.01	

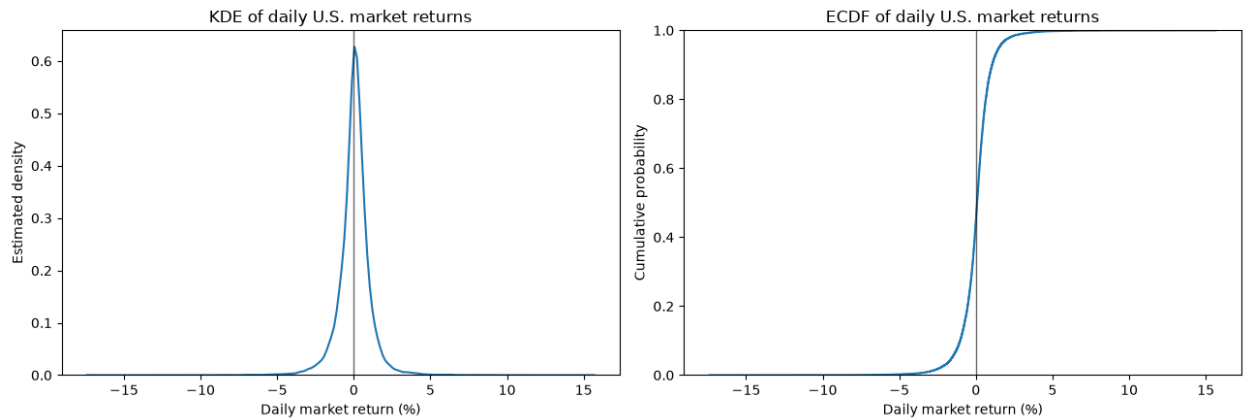
1. Plot the KDE and ECDF of daily market returns.

```
In [64]: fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

sns.kdeplot(
    data=df,
    x="market_return_pct",
    cut=0,
    ax=axes[0],
)
axes[0].axvline(0, color="black", linewidth=1, alpha=0.6)
axes[0].set(
    title="KDE of daily U.S. market returns",
    xlabel="Daily market return (%)",
    ylabel="Estimated density",
)

sns.ecdfplot(
    data=df,
    x="market_return_pct",
    ax=axes[1],
)
axes[1].axvline(0, color="black", linewidth=1, alpha=0.6)
axes[1].set(
    title="ECDF of daily U.S. market returns",
    xlabel="Daily market return (%)",
    ylabel="Cumulative probability",
)

plt.tight_layout()
plt.show()
```



- For each calendar month, compute the **best** (max) and **worst** (min) daily return — the first and n -th order statistics of that month's trading days. Plot the KDE of the monthly best and worst returns, overlaid.

```
In [65]: monthly = (
    df.groupby(["year", "month"], as_index=False)
    .agg(
        month_start=("month_start", "first"),
        trading_days=("date", "size"),
        worst_return_pct=("market_return_pct", "min"),
        best_return_pct=("market_return_pct", "max"),
        median_return_pct=("market_return_pct", "median"),
    )
)

# Put losses and gains on the same positive-magnitude scale.
monthly["worst_loss_pct"] = -monthly["worst_return_pct"]

print(f"{len(monthly):,} calendar months")
print(
    f"Trading days per month: {monthly['trading_days'].min()} "
    f"to {monthly['trading_days'].max()}"
)
```

```
monthly.head()
```

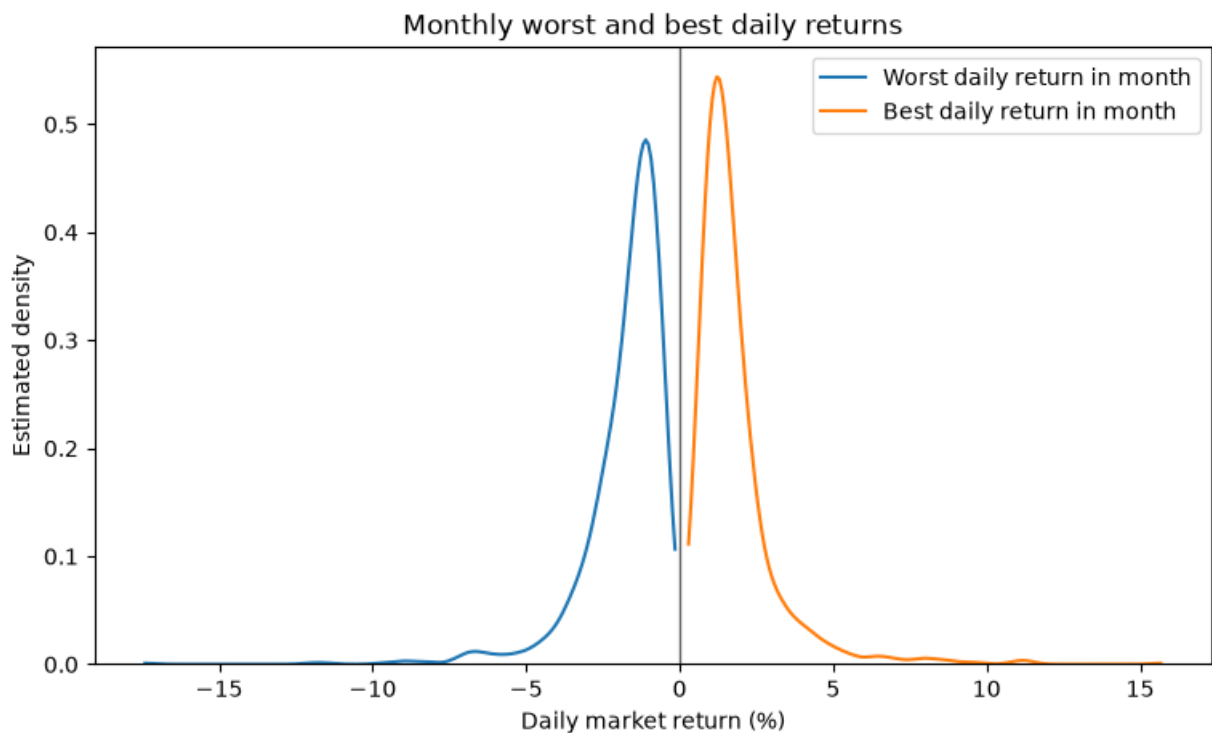
1,201 calendar months

Trading days per month: 15 to 27

```
Out[65]:
```

	year	month	month_start	trading_days	worst_return_pct	best_return_pct	median_return_pct	worst_loss_pct
0	1926	7	1926-07-01	25	-0.72	1.10	0.18	0.72
1	1926	8	1926-08-01	26	-1.39	0.85	0.32	1.39
2	1926	9	1926-09-01	24	-0.98	0.91	-0.03	0.98
3	1926	10	1926-10-01	25	-1.81	1.53	-0.09	1.81
4	1926	11	1926-11-01	24	-0.62	0.67	0.19	0.62

```
In [66]: plt.figure(figsize=(9, 5))
sns.kdeplot(
    x=monthly["worst_return_pct"],
    label="Worst daily return in month",
    cut=0
)
sns.kdeplot(
    x=monthly["best_return_pct"],
    label="Best daily return in month",
    cut=0
)
plt.axvline(0, color="black", linewidth=1, alpha=0.6)
plt.xlabel("Daily market return (%)")
plt.ylabel("Estimated density")
plt.title("Monthly worst and best daily returns")
plt.legend()
plt.show()
```



Describe the symmetry (or lack of it) between the best- and worst-day densities. Compare them to the overall populations of daily returns. Use code as appropriate to make these comparisons, and then write out your answer in a markdown cell.

```
In [67]: print(f"Avg Best: {monthly['best_return_pct'].mean()}")
print(f"Avg Worst: {monthly['worst_return_pct'].mean()}")
```

```

print(f"STD Best: {monthly['best_return_pct'].std()}")
print(f"STD Worst: {monthly['worst_return_pct'].std()}")

print(f"All positive months: {len(monthly[monthly['best_return_pct'] <= 0].index)}")
print(f"All negative months: {len(monthly[monthly['worst_return_pct'] >= 0].index)}")

```

```

Avg Best: 1.8074604496253124
Avg Worst: -1.7778684429641964
STD Best: 1.390836545144491
STD Worst: 1.397024022465251
All positive months: 0
All negative months: 0

```

The distributions are close to symmetrical, but aren't. The best daily returns has a higher density peak, and it's peak is closer to 0. This alone would suggest that the magnitude of the average best daily return is smaller than the magnitude of average of the worst daily returns, but since the average magnitude for the bests is higher than the worsts, the best density curve must have more high magnitude outlier variables, which seems to have based on the plot. The variance for the bests and worsts is very close to being the same, which matches what visually seems to be two very similarly shaped curves. Every month had at least once positive return and at least one negative return day.

3. Losses and gains are on opposite sides of zero, which makes them hard to compare directly. Put them on the same scale by looking at the *magnitude* of the worst loss (`-worst_return_pct`) against the magnitude of the best gain (`best_return_pct`). Plot both the KDE and ECDF.

In [68]: `fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))`

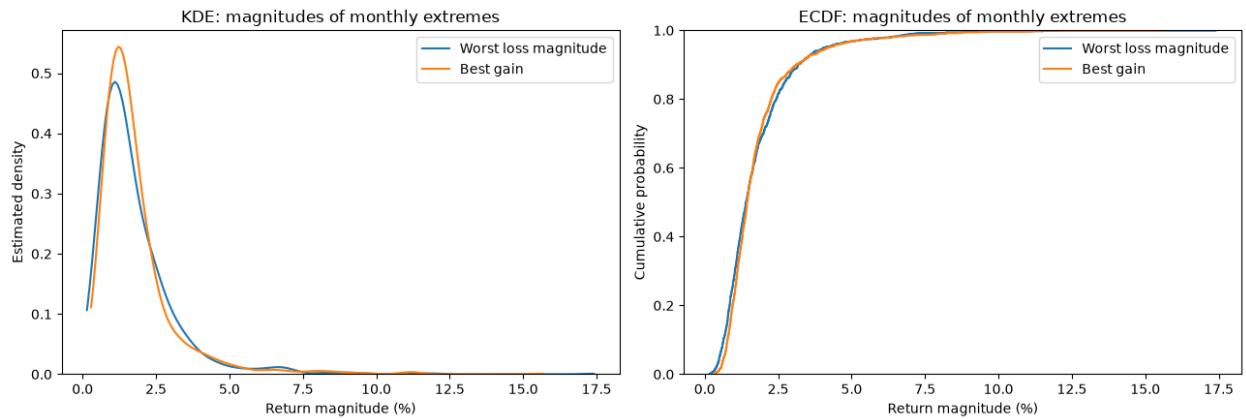
```

sns.kdeplot(
    x=monthly["worst_loss_pct"],
    label="Worst loss magnitude",
    cut=0,
    ax=axes[0],
)
sns.kdeplot(
    x=monthly["best_return_pct"],
    label="Best gain",
    cut=0,
    ax=axes[0],
)
axes[0].set(
    title="KDE: magnitudes of monthly extremes",
    xlabel="Return magnitude (%)",
    ylabel="Estimated density",
)
axes[0].legend()

sns.ecdfplot(
    x=monthly["worst_loss_pct"],
    label="Worst loss magnitude",
    ax=axes[1],
)
sns.ecdfplot(
    x=monthly["best_return_pct"],
    label="Best gain",
    ax=axes[1],
)
axes[1].set(
    title="ECDF: magnitudes of monthly extremes",
    xlabel="Return magnitude (%)",
    ylabel="Cumulative probability",
)
axes[1].legend()

plt.tight_layout()
plt.show()

```



Are large losses more extreme than large gains, or about the same:

Large losses are roughly as extreme as the large gains. Both of the lines on the ECDF and KDE plots follow roughly the same shape, especially at the high return magnitudes (> 5.0%). The main differences in the curves happen at the lower magnitude ranges.

4. Compute the median, 75th, 90th, 95th, and 99th percentile of both magnitudes from question 3. In what fraction of months does the worst loss exceed the best gain in magnitude?

```
In [69]: tail_summary = pd.DataFrame(
    {
        "Worst loss magnitude": monthly["worst_loss_pct"],
        "Best gain": monthly["best_return_pct"],
    }
).quantile([0.50, 0.75, 0.90, 0.95, 0.99]).T

tail_summary.columns = ["median", "75%", "90%", "95%", "99%"]

print(
    "Fraction of months in which the worst loss exceeded the best gain: "
    f"{(monthly['worst_loss_pct'] > monthly['best_return_pct']).mean():.1%}"
)
tail_summary.round(2)
```

Fraction of months in which the worst loss exceeded the best gain: 44.8%

```
Out[69]:
```

	median	75%	90%	95%	99%
Worst loss magnitude	1.41	2.21	3.18	4.06	6.97
Best gain	1.44	2.04	3.12	4.21	7.93

What does this tail comparison say about the risk of losses vs. gains:

This tells us that the most uncommonly large magnitude gain / loss days (95%+) are going to have larger magnitudes when there is a gain than when there is a loss. The worst loss in a month is generally going to have smaller magnitude than the best gain in a month.

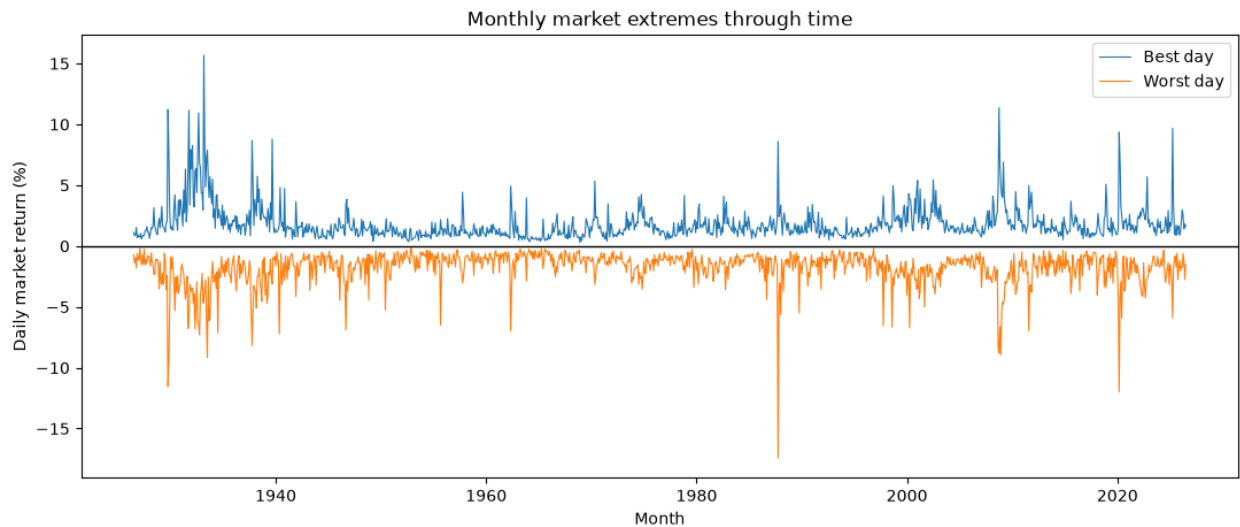
5. Plot the monthly best and worst daily returns over time. Do the extremes look stable across this ~100-year history, or are there periods where they're much larger? Can you identify any famous market events?

```
In [70]: plt.figure(figsize=(13, 5))
plt.plot(
    monthly["month_start"],
    monthly["best_return_pct"],
    linewidth=0.8,
```

```

    label="Best day",
)
plt.plot(
    monthly["month_start"],
    monthly["worst_return_pct"],
    linewidth=0.8,
    label="Worst day",
)
plt.axhline(0, color="black", linewidth=1)
plt.xlabel("Month")
plt.ylabel("Daily market return (%)")
plt.title("Monthly market extremes through time")
plt.legend()
plt.show()

```



Do upside and downside risk tend to go together (volatility), or does the market exhibit distinct periods in which the good shocks are concentrated and the bad shocks are concentrated? What implication does this have for investing?:

It does seem like whenever there is a spike in the highest daily market return for a month, there is also a spike in the lowest daily market return for that same month (or for a neighboring month, hard to tell from this plot). There are definitely periods where there is increased volatility for both the best and worst days in a month, which seem to be correlated with financial crises (the great depression in the 1930s, the 2008 housing crisis, COVID in 2020, etc). There don't seem to be any periods with more extreme best days without there being more extreme worst days, and vice versa.

Wrap-up

This week has focused on kernel density estimation, particularly estimating the PDF from data. This lab takes that a step further: We can estimate and quantify the densities and distributions of random variables that build off of observed phenomena. What is the difference between the ECDF and KDE? Explain how these estimations today give us different information than an ECDF or KDE plot of the underlying data, but quantify information about the behavior of a different random variable.

There are connections in lots of the data that get lost when we look at the ECDF or KDE for the full dataset. For the weather data, looking at a holistic ECDF or KDE shows us the range of possible values, and their relative likelihoods, but doesn't contextualize the data the way an ECDF / KDE for the median / maximum values for grouped data does. Looking at these plots for different, but related, random variables allows us to make more useful plots / interpretations from the data than just looking at the data as a whole.